

CO3-AT2 – Question 39

Case Study: Satellite Image Processing (Matrix Multiplication)

Name: Talari Vishnuvardhan

Register No: 192572091

Subject: Design and Analysis of Algorithms

Faculty: Dr. F. Mary Harin Fernandez

Question No: 39

Title

Efficient Satellite Image Processing using Divide and Conquer Matrix Multiplication

1. Problem Statement

Satellite images are very large and contain millions of pixels. Processing these images requires performing several matrix operations such as image enhancement, rotation, filtering, and transformation. Traditional matrix multiplication becomes slow when image sizes increase.

The objective of this case study is to improve the efficiency of satellite image processing using the Divide and Conquer Matrix Multiplication technique. The solution should reduce execution time while maintaining accuracy.

2. Objectives

- Study satellite image processing using matrix multiplication.
- Implement Divide and Conquer Matrix Multiplication.
- Compare its performance with the normal approach.
- Analyze challenges in processing large satellite images.
- Suggest optimization techniques.

3. Understanding the Case Study

Satellite images are represented as matrices.

Example:

Image A

1 2

3 4

Transformation Matrix B

5 6

7 8

The multiplication of these matrices performs image transformations such as scaling, rotation, and enhancement.

For high-resolution images (4096×4096 or larger), normal multiplication becomes computationally expensive.

4. Key Challenges

Challenge 1: Large Image Size

Satellite images may contain millions of pixels requiring huge computations.

Challenge 2: High Memory Usage

Large matrices consume significant RAM during processing.

Challenge 3: Longer Processing Time

Traditional multiplication requires many arithmetic operations.

Challenge 4: Computational Complexity

Normal matrix multiplication has

Time Complexity

$O(n^3)$

which becomes inefficient for large datasets.

Challenge 5: Real-Time Processing

Disaster monitoring and weather forecasting require quick image processing.

5. Divide and Conquer Approach

Instead of multiplying the whole matrix directly,

Divide and Conquer performs the following steps:

Step 1

Divide Matrix A into four equal parts

A11 A12

A21 A22

Step 2

Divide Matrix B similarly

B11 B12

B21 B22

Step 3

Recursively multiply smaller matrices.

Step 4

Combine all results into the final matrix.

This significantly improves efficiency and enables parallel computation.

6. Algorithm

MatrixMultiply(A,B)

If matrix size == 1

Return $A \times B$

Else

Divide A into four submatrices

Divide B into four submatrices

Compute

$C_{11} = A_{11} \times B_{11} + A_{12} \times B_{21}$

$C_{12} = A_{11} \times B_{12} + A_{12} \times B_{22}$

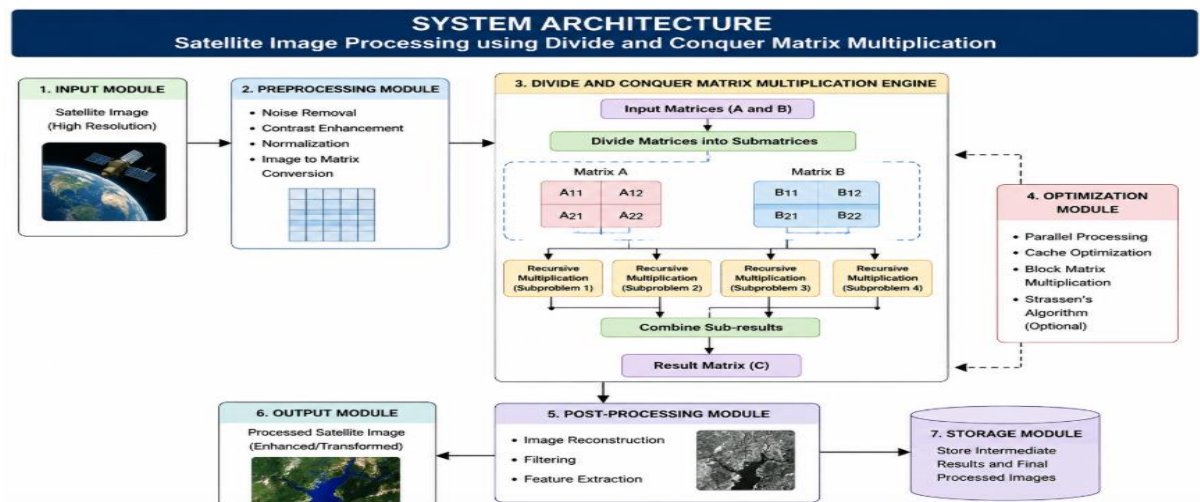
$C_{21} = A_{21} \times B_{11} + A_{22} \times B_{21}$

$C_{22} = A_{21} \times B_{12} + A_{22} \times B_{22}$

Combine C11,C12,C21,C22

Return C

7. System Architecture



8. Optimized Techniques

Technique 1

Divide and Conquer Matrix Multiplication

Reduces computation by recursively processing smaller matrices.

Technique 2

Parallel Processing

Different processors calculate different submatrices simultaneously.

Technique 3

Cache Optimization

Keeps frequently used data in cache memory for faster access.

Technique 4

Block Matrix Multiplication

Processes matrices in smaller blocks, improving cache performance.

Technique 5

Strassen's Algorithm

Reduces multiplication operations.

Time Complexity

$$O(n^{2.81})$$

which is better than $O(n^3)$.

9. Python Program

```
1 import numpy as np
2 def divide_and_conquer(A, B):
3     n = len(A)
4     if n == 1:
5         return A * B
6     mid = n // 2
7     A11 = A[:mid, :mid]
8     A12 = A[:mid, mid:]
9     A21 = A[mid:, :mid]
10    A22 = A[mid:, mid:]
11    B11 = B[:mid, :mid]
12    B12 = B[:mid, mid:]
13    B21 = B[mid:, :mid]
14    B22 = B[mid:, mid:]
15    C11 = divide_and_conquer(A11, B11) + divide_and_conquer(A12, B21)
16    C12 = divide_and_conquer(A11, B12) + divide_and_conquer(A12, B22)
17    C21 = divide_and_conquer(A21, B11) + divide_and_conquer(A22, B21)
18    C22 = divide_and_conquer(A21, B12) + divide_and_conquer(A22, B22)
19    top = np.hstack((C11, C12))
20    bottom = np.hstack((C21, C22))
21    A = np.array([[1,2],[3,4]])
22    B = np.array([[5,6],[7,8]])
23    print("Matrix A")
24    print(A)
25    print("\nMatrix B")
26    print(B)
27    print("\nResult")
28    print(divide_and_conquer(A,B))
```

OUTPUT

```
Matrix A
[[1 2]
 [3 4]]

Matrix B
[[5 6]
 [7 8]]

Result
[[19 22]
 [43 50]]
```

11. Performance Analysis

Method	Time Complexity	Memory Usage	Performance
Normal Matrix Multiplication	$O(n^3)$	Medium	Slower
Divide and Conquer	$O(n^3)$ (supports parallel execution)	Medium	Faster in practice
Strassen Algorithm	$O(n^{2.81})$	Slightly Higher	Fastest for large matrices

12. Advantages

- Faster processing of large satellite images.
- Supports parallel computing.
- Better cache utilization.
- Suitable for high-resolution images.
- Reduces execution time for large datasets.
- Improves scalability.

13. Limitations

- Recursive implementation increases function calls.
- More memory is required for recursion.
- Small matrices may not benefit significantly.
- Strassen's algorithm is more complex to implement.
- Hardware limitations affect performance.

14. Applications

- Weather forecasting
- Disaster management

- Forest monitoring
- Military surveillance
- Agricultural monitoring
- Urban planning
- Environmental monitoring
- Remote sensing

15. Analysis

The Divide and Conquer technique improves the efficiency of satellite image processing by splitting large matrices into smaller subproblems. Recursive multiplication allows better utilization of processor resources and supports parallel execution. Compared to the traditional method, it reduces processing time for large datasets and provides better scalability. For extremely large images, optimized methods such as Strassen's algorithm further reduce computational complexity, making them suitable for real-time satellite image analysis.

16. Conclusion

Satellite image processing involves intensive matrix computations that become expensive as image sizes grow. Divide and Conquer Matrix Multiplication improves performance by recursively dividing matrices into smaller blocks and combining the results efficiently. The approach enhances scalability, enables parallel processing, and reduces practical execution time. For very large datasets, combining Divide and Conquer with advanced algorithms like Strassen's method provides an effective solution for modern satellite image processing systems.

17. References

1. Cormen, T. H., *Introduction to Algorithms*, MIT Press.
2. Ellis Horowitz, *Fundamentals of Computer Algorithms*.
3. S. Dasgupta, *Algorithms*.
4. Rafael C. Gonzalez & Richard E. Woods, *Digital Image Processing*.
5. IEEE Digital Library – Research papers on Matrix Multiplication and Satellite Image Processing.